

CMSC216: x86-64 Control Flow

Chris Kauffman

Last Updated:

Thu Oct 8 09:26:22 AM EDT 2026

Logistics

Announcements

Control Flow in Assembly and the Instruction Pointer

Instruction Pointer Register

- ▶ **%rip: special register** (not general purpose) referred to as the **Instruction Pointer** or Program Counter
- ▶ **%rip** contains main memory address of next assembly instruction to execute
- ▶ After executing an instruction, **%rip** automatically updates to the subsequent instruction
OR in a Jump instruction, **%rip** changes non-sequentially
- ▶ **Do not** add/subtract with **%rip** via `addq/subq`: **%rip** automatically updates after each instruction

Jump Instructions

- ▶ **Labels** in assembly indicate jump targets like `.LOOP`:
- ▶ **Unconditional Jump**: always jump to a new location by changing **%rip** non-sequentially
- ▶ **Comparison / Test**: Instruction, sets EFLAGS bits indicating relation between registers/values (greater, less than, equal)
- ▶ **Conditional Jump**: Jumps to a new location if certain bits of EFLAGS are set by changing **%rip** non-sequentially; otherwise continues sequential execution

Exercise: Loop Sum with Instruction Pointer (rip)

- ▶ Can see direct effects on `rip` in disassembled code <MINTED>
- ▶ `rip` increases corresponding to instruction length
- ▶ Jumps include address for next `rip`

<MINTED>

Disassembling Binaries

- ▶ Binaries hard to read on their own
- ▶ Many tools exist to work with them, notably `objdump` on Unix
- ▶ Can **disassemble** binary: show “readable” version of contents

<MINTED>

FLAGS: Condition Codes Register

- ▶ Most CPUs have a special register with “flags” for various conditions: each bit is True/False for a specific condition
- ▶ In x86-64 this register goes by the following names

Name	Width	Notes
FLAGS	16-bit	Most important bits in first 16
EFLAGS	32-bit	Name shown in GDB
RFLAGS	64-bit	Not used normally

- ▶ Bits in FLAGS register are **automatically** set based on results of other operations like addition / subtraction / etc.

Bit	Abbrev	Name	Description
0	CF	Carry flag	Set if last op caused Unsigned overflow
6	ZF	Zero flag	Set if last op yielded a 0 result
7	SF	Sign flag	Set if last op yielded a negative
8	TF	Trap flag	Used by gdb to stop after one ASM instruction
9	IF	Interrupt flag	1: handle hardware interrupts, 0: ignore them
11	OF	Overflow flag	Set if last op caused SIGNED overflow/underflow

Comparisons and Tests

Set the EFLAGS register by using comparison instructions

Name	Instruction	Examples	Notes
Compare	cmpX B, A	cmpl \$1,%eax	Like if(eax > 1){...}
	Like: A - B	cmpq %rsi,%rdi	Like if(rdi > rsi){...}
Test	testX B, A	testq %rcx,%rdx	Like if(rdx & rcx){...}
	Like: A & B	testl %rax,%rax	Like if(rax){...}

- ▶ Immediates like \$2 must be the first argument B
- ▶ B, A are NOT altered with cmp/test instructions
- ▶ EFLAGS register IS changed by cmp/test to indicate less than, greater than, 0, etc.

<MINTED>

Jump Instruction Summary

All control structures implemented using combination of Compare/Test + Jump instructions.

<i>Instruction</i>	<i>Jump Condition</i>	<i>FLAGS</i>
jmp LAB	Unconditional jump	-
je LAB	Equal / zero	ZF
jz LAB		ZF
jne LAB	Not equal / non-zero	!ZF
jnz LAB		!ZF
js LAB	Negative ("signed")	SF
jns LAB	Nonnegative	!SF
jg LAB	Greater-than signed	!(SF xor OF) and !ZF
jge LAB	Greater-than-equal signed	!(SF xor OF)
jl LAB	Less-than signed	(SF xor !OF)
jle LAB	Less-than-equal signed	(SF xor !OF) or !ZF
ja LAB	Above unsigned	!CF and !ZF
jae LAB	Above-equal unsigned	!CF
jb LAB	Below unsigned	CF and !ZF
jbe LAB	Below-equal unsigned	CF
jmp *OPER	Unconditional jump to variable address	-

Examine: Compiler Comparison Inversion

- ▶ Often compiler inverts comparisons
- ▶ `i < n` becomes `cmpX / jge` (jump greater/equal)
- ▶ `i == 0` becomes `cmpX / jne` (jump not equal)
- ▶ This allows “true” case to fall through immediately <MINTED>
- ▶ Depending on structure, may have additional jumps
 - ▶ `if(){ .. }` usually has a single jump
 - ▶ `if(){} else {}` may have a couple

Logical And / Or in Assembly

Logical boolean operators like `a && b` and `x || y` translate sequences of compare/test instructions followed by conditional jumps. See `andcond_asm.s` and `nestedcond_asm.s`

<MINTED>

C Boolean expressions may “short circuit”: never execute code associated with later parts of the condition if early part resolves conditional

<MINTED>

Exercise: The test Instruction

<MINTED>

- ▶ `testl %eax,%eax` uses bitwise AND to examine a register
- ▶ Selected by compiler to check for zero, NULL, negativity, etc.
- ▶ Followed by `je / jz / jne / jnz / js / jns`
- ▶ **Trace the execution**
- ▶ Determine final value in `%eax`
- ▶ Will demo `jmp_tests_asm.s` in GDB to show how EFLAGS changes

Answers: The test Instruction

<MINTED>

<MINTED>

cmov Family: Conditional Moves

- ▶ Instruction family which copies data conditioned on FLAGS¹
- ▶ Can limit jumping in simple assignments

<MINTED>

- ▶ Note flags set on **all Arithmetic Operations**
- ▶ cmpX is like subQ: both set FLAG bits the same
- ▶ Greater than is based on the SIGN flag indicating subtraction would be negative allowing the following:

<MINTED>

¹Other architectures like ARM have conditional versions of many instructions like `addlt r1, r2, r3`; RISC V ditches the FLAGS register in favor of jumps based on comparisons like `BLT x0, x1, LOOP`

Procedure Calls

Have seen basics so far:

<MINTED>

Need several additional notions

- ▶ Control Transfer to called function?
- ▶ Return back to calling function?
- ▶ Stack alignment and conventions
- ▶ Register conventions

Procedure Calls Return to Arbitrary Locations

- ▶ `call` instructions always transfer control to start of `return_seven` at line 4/5, like `jmp` instruction which modifies `%rip`
- ▶ `ret` instruction at line 6 must transfer control to **different locations**
 1. `call`-ed at line 11
 `ret` to line 12
 2. `call`-ed at line 17
 `ret` to line 18

`ret` cannot be a normal `jmp`
- ▶ To enable return to multiple places, record a **Return Address** when `call`-ing, use it when `ret`-urning

<MINTED>

call / ret with Return Address in Stack

call Instruction

1. Push the “caller” **Return Address** onto the stack
Return address is for instruction after call
2. Change rip to first instruction of the “callee” function

ret Instruction

1. Set rip to Return Address at top of stack
2. Pop the Return Address off to shrink stack

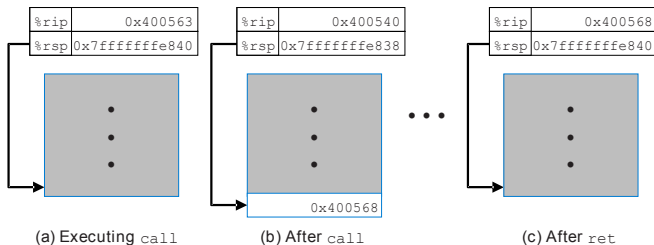


Figure: Bryant/O'Hallaron Fig 3.26 demonstrates call/return in assembly

return_seven_asm.s 1/2: Control Transfer with call

<MINTED>

return_seven_asm.s 2/2: Control Transfer with ret

<MINTED>

Warning: `%rsp` is important for returns

- ▶ When a function is about to return `%rsp` MUST refer to the memory location of the return address
- ▶ `ret` uses value pointed to `%rsp` as the return address
- ▶ Segmentation Faults often occur if `%rsp` is NOT the return address: attempt to fetch/execute instructions out of bounds
- ▶ Stack is often used to store local variables, stack pointer `%rsp` is manipulated via `pushX` / `subq` instructions to grow the stack.
- ▶ Before returning MUST shrink stack and restore `%rsp` to its original value via `popX` / `addq` instructions
- ▶ There are computer security issues associated stack-based return value we will discuss later

Messing up the Return Address

<MINTED>

<MINTED>

<MINTED>

Valgrind reports like this often indicate failure to restore the stack pointer as happened here. If the stack grows, shrink it before returning.

Stack Alignment

- ▶ According to the strict x86-64 ABI, must align `rsp` (stack pointer) to 16-byte boundaries when calling functions
- ▶ Will often see arbitrary pushes or subtractions to align
 - ▶ Functions called with 16-byte alignment
 - ▶ `call` pushes 8-byte Return Address on the stack
 - ▶ At minimum, must grow stack by 8 bytes to `call` again
- ▶ `rsp` changes must be undone prior to return
<MINTED>
- ▶ Skipping stack alignment works on some machines and breaks on others due to differing processor internals
 - "I passed all the tests on GRACE but a bunch failed on Gradescope! What gives?"*
 - Common lament of past students*

x86-64 Register/Procedure Convention

- ▶ Used by Linux/Mac/BSD/General Unix
- ▶ Params and return in registers if possible

Parameters and Return

RetVal	rax / eax / ax / al
Arg 1	rdi / edi / di / dil
Arg 2	rsi / esi / si / sil
Arg 3	rdx / edx / dx / dl
Arg 4	rcx / ecx / cx / cl
Arg 5	r8 / r8d / r8w / r8b
Arg 6	r9 / r9d / r9w / r9b
Arg 7	Push into the stack
Arg 8	Push into the stack
...	...

C function prototype indicates number, order, type of args so it is known which registers args will be in

<MINTED>

Caller/Callee Save

Caller save registers: alter freely

rax rcx rdx rdi rsi
r8 r9 r10 r11 # 9 regs

Callee save registers: must restore these before returning

rbx rbp r12 r13 r14
r15 # 6 regs

Stack Pointer: special considerations discussed in detail

rsp # 1 reg

Caller and Callee Save Register Mechanics

CalleR Save Regs

May all change across function call boundaries.

Not a problem for **Leaf Functions** which do not call any other funcs

CalleE Save Regs

Have the same values in them after a function call

Using them requires saving their original values in the stack and restoring them

`sumrange_asm.s`

Full example of callee save regs like `sumrange_c.c`

<MINTED>

Pushing and Popping the Stack

- ▶ If local variables or callee save regs are needed on the stack, can use push / pop for these
- ▶ Push and Pop Instructions are compound: manipulate %rsp and move data in single instruction

pushX data	Grow Stack, store data at top
pushq %rax	Like: subq \$8,%rsp; movq %rax, (%rsp)
pushl \$25	Like: subq \$4,%rsp; movq \$25, (%rsp)
popX data	Shrink Stack, restore data from it
popl %edi	Like: movl (%rsp),%edi; addq \$4,%rsp;
popq %rax	Like: movq (%rsp),%rax; addq \$8,%rsp;

<MINTED>

Exercise: Local Variables which need an Address

Compare code in files

- ▶ `swap_pointers.c` : familiar C code for swap via pointers
- ▶ `swap_pointers_asm.s` : hand-coded assembly version

Determine the following

1. Where are `main()` local variables `x,y` stored in assembly?
2. Where does the assembly version “grow” the stack?
3. How are the values in `main()` passed as arguments to `swap_ptr()`?
4. Where are `swap_ptr()` local variables stored in assembly?
5. Where does the assembly version “shrink” the stack?

Exercise: Local Variables which need an Address

1. Where are `main()` local variables `x,y` stored in assembly?
2. Where does the assembly version "grow" the stack?
3. How are the values in `main()` passed as arguments to `swap_ptr()`?
4. Where are `swap_ptr()` local variables stored in assembly?
5. Where does the assembly version "shrink" the stack?

<MINTED>

<MINTED>

Answers: Local Variables which need an Address

1. Where are `main()` local variables `x,y` stored in assembly?
2. Where does the assembly version “grow” the stack?
3. How are the values in `main()` passed as arguments to `swap_ptr()`?

<MINTED>

<MINTED>

4. Where are `swap_ptr()` local variables stored in assembly?
Stored entirely in registers; args `%rdi,%rsi` are `a,b` while `tmp` is spread across regs `%eax,%edx`
5. Where does the assembly version “shrink” the stack?

<MINTED>

Diagram of Stack Variables

- ▶ Compiler determines if local variables go on stack
- ▶ If so, calculates location as `rsp + offsets`

<MINTED>

<MINTED>

<MINTED>

Summary of Procedure Calls: `ABC()` calls `XYZ()`

<code>ABC()</code>	Caller	<code>callq XYZ</code>	# <code>ABC</code> to <code>XYZ</code>
<code>XYZ()</code>	Callee	<code>retq</code>	# <code>XYZ</code> to <code>ABC</code>

1. `ABC()` “saves” any Caller Save registers it needs by either copying them into Callee Save registers or pushing them into the stack
2. `ABC()` places up to 6 arguments in `%rsi`, `%rdi`, `%rdx`, ..., remaining arguments in stack
3. `ABC()` ensures that stack is “aligned”: `%rsp` contains an address that is evenly divisible by 16
4. `ABC()` issues the `callq ABC` instruction which (1) grows the stack by subtracting 8 from `%rsp` and copies a return address to that location and (2) changes `%rip` to the starting address of `func`
5. `XYZ()` now has control: `%rip` points to first instruction of `XYZ()`
6. `XYZ()` may issue `pushX val` instructions or `subq N,%rsp` instructions to grow the stack for local variables
7. `XYZ()` may freely change Caller Save registers BUT Callee Save registers it changes must be restored prior to returning.
8. `XYZ()` must shrink the stack to its original position via `popX %reg` or `addq N,%rsp` instructions before returning.
9. `XYZ()` sets `%rax` / `%eax` / `%ax` to its return value if any.
10. `XYZ()` finishes, issues the `retq` instruction which (1) sets the `%rip` to the 8-byte return address at the top of the stack (pointed to by `%rsp`) and (2) shrinks the stack by doing `addq $8,%rsp`
11. `ABC()` function now has control back with `%rip` pointing to instruction after `call XYZ`; may have a return value in `%rax` register
12. `ABC()` must assume all Caller Save registers have changed

History: Base Pointer `rbp` was Previously Special Use

<MINTED>

- ▶ 32-bit x86 / IA32 assembly used `rbp` and `rsp` to describe stack frames
- ▶ All function args pushed onto the stack when calling, changes both Stack/Base Pointer
- ▶ x86-64: optimizes `%rbp` for general purpose use, points at stack only in special situations

<MINTED>